

LAB

03

IMPLEMENTING JUMP AND RETURN INSTRUCTIONS IN RISC- V ASSEMBLY LANGUAGE

Overview

- Function Calls
- Stacks
- Environment Calls

Function Call

- Used to avoid repetition of the same piece of

Unconditional branch	Jump and link	<code>jal x1, 100</code>	<code>x1 = PC+4; go to PC+100</code>	PC-relative procedure call
	Jump and link register	<code>jalr x1, 100(x5)</code>	<code>x1 = PC+4; go to x5+100</code>	Procedure return; indirect call

1. jal (jump & link) → used for function calls

`jal rd, offset`

→ Can be an immediate value or a label

- Jumps to target address and stores return address (PC+4) in a register (typically x1 - ra).

`jal x1, L1` # jump to label 'L1', and store the return address of next instruction (PC+4) in 'x1' register

Function Call



Unconditional branch	Jump and link	<code>jal x1, 100</code>	<code>x1 = PC+4; go to PC+100</code>	PC-relative procedure call
	Jump and link register	<code>jalr x1, 100(x5)</code>	<code>x1 = PC+4; go to x5+100</code>	Procedure return; indirect call

2. *jalr (jump & link register)* → used for indirect jumps (return from a function)

jalr x1, 0(x2) # jump to target address stored in 'x2 + offset (0)' and stores return address (PC+4) in 'x1'

- Jumps to target address specified in a source register (+ offset value) and stores return address in another destination register.

Function Call



Unconditional branch	Jump and link	<code>jal x1, 100</code>	<code>x1 = PC+4; go to PC+100</code>	PC-relative procedure call
	Jump and link register	<code>jalr x1, 100(x5)</code>	<code>x1 = PC+4; go to x5+100</code>	Procedure return; indirect call

2. *jalr (jump & link register)* → used for indirect jumps (return from a function)

- Since it is a return instruction, and not a function call. So usually there is no **need** to save a new return address. So destination register is often x0 where the new PC value is overwritten to zero to intentionally throw away the return address.

jalr x0, 0(x1) # jump to target address stored in 'x1' with offset 0, and store
return address in 'x0'

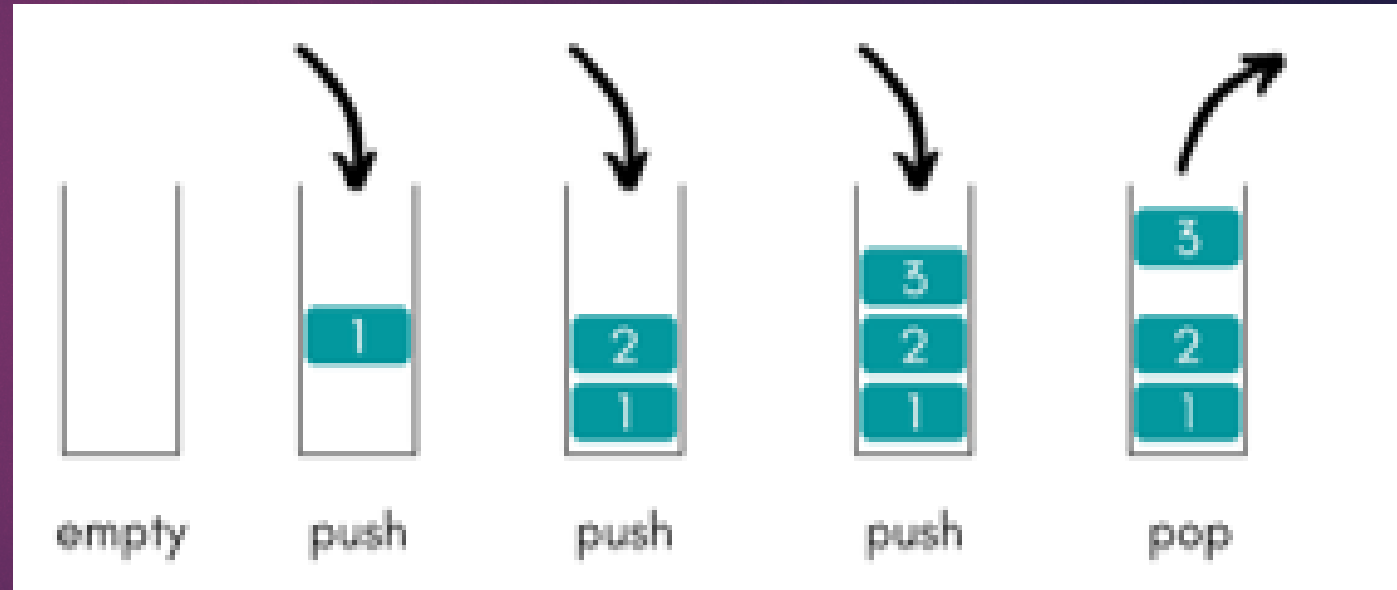
RISC-V Registers

Register	ABI Name	Description	Saved by
x0	zero	zero register (hardwired zero)	
x1	ra	return address	Caller
x2	sp	stack pointer	Callee
x3	gp	global pointer	
x4	tp	thread pointer	
x5–7	t0–2	temporary registers	Caller
x8	s0 / fp	saved register / frame pointer	Callee
x9	s1	saved register	Callee
x10–11	a0–1	function arguments / return values	Caller
x12–17	a2–7	function arguments	Caller
x18–27	s2–11	saved registers	Callee
x28–31	t3–6	temporary registers	Caller

Note: If more registers (other than 8 argument registers (x10-17)) needed >> use 'saved registers – x8, x9, x18-27'

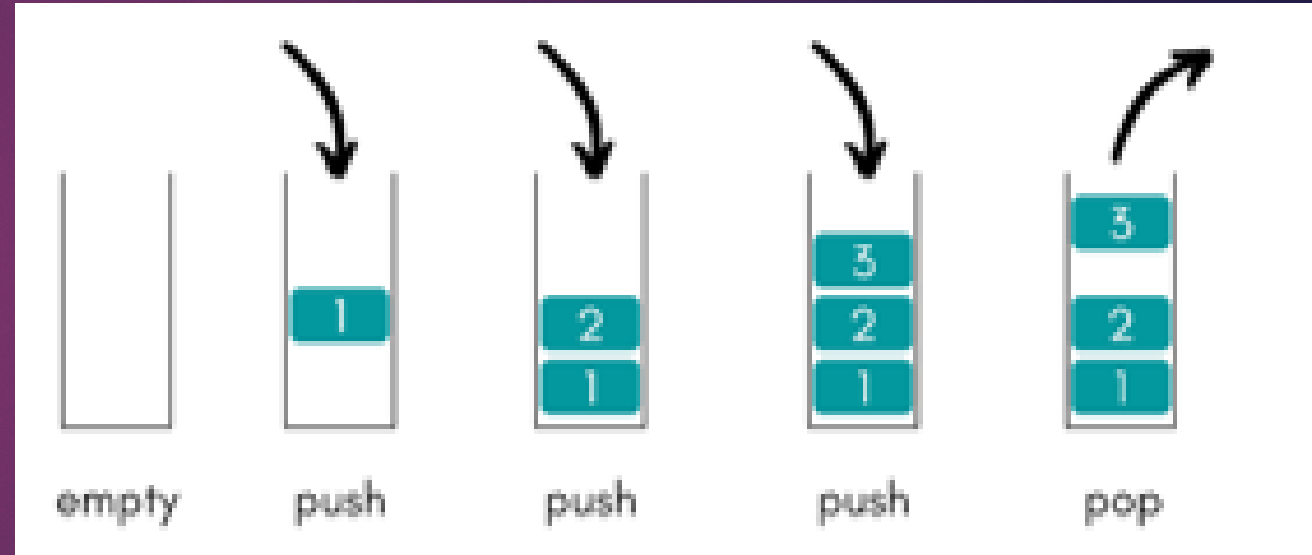
Stack

- If there is a requirement of more registers (other than 8 argument registers i.e. x10 - 17), we need to make a stack and use '*saved registers i.e. x8 - 9, x18 - 27*'.
- **Pushing** values onto the stack in RISC-V means temporarily saving those values (already saved in registers) in memory.
- This is typically done to preserve register values during function calls, so they can be restored later when needed.



Stack

- **Stack:** Data structure for *spilling* registers organized as a **Last-In-First-Out (LIFO)** queue.
- **Stack Pointer:** A value denoting the most recently allocated address in a stack that shows where the next procedure should place the registers to be spilled, or where the old register values can be found.
- Typically register 'x29' / 'sp' is used.
- If the value stored in the register is a 'Word' (4 Bytes), then 'sp' is adjusted by 4 Bytes for each register that is saved/restored.



Example: Stacks – push in the

values

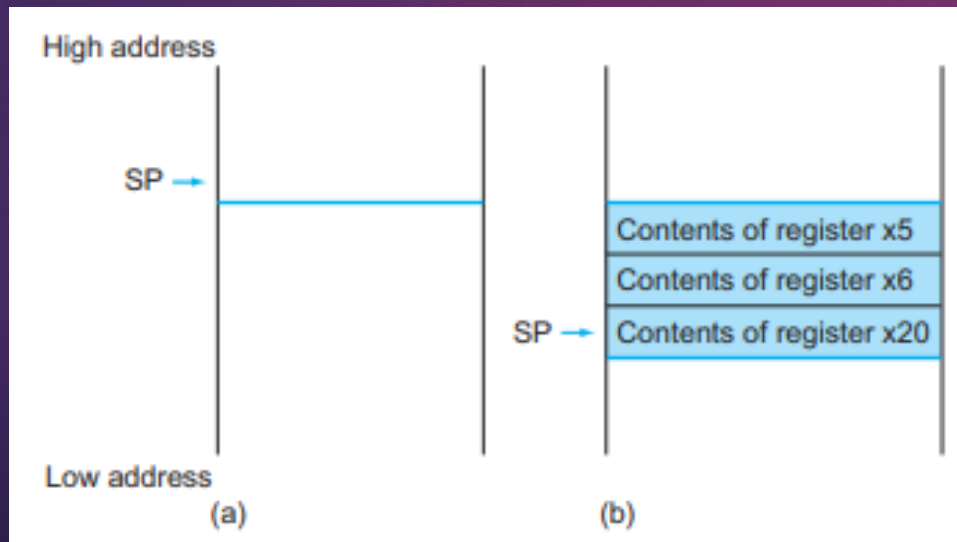
- Data type of the value being pushed in & popped out: **word – 4 Bytes**
- Requirements: x5, x6 & x20 is needed for further calculation (assume these registers already have some valuable values loaded into them). 'Pushing' old values onto stack by creating space for 3 Word (12 Bytes) on stack & store them.

addi sp, sp, -12 Each item is of 4 Bytes: $4 + 4 + 4 = 12$ Bytes
#adjust stack to make room for 3 items (x5, x6, x20 registers) – space for 12 Bytes.

sw x5, 8(sp) #value of register 'x5' is stored 8 Bytes from the current stack pointer (sp). This means that the location for x5 is at the highest address within the allocated space.

sw x6, 4(sp) # value of 'x6' is stored 4 Bytes from the current sp. It is placed just below x5.

sw x20, 0(sp) # value of 'x20' is stored at the address currently pointed to by sp. This is the lowest address within the 12-Byte block of stack space.



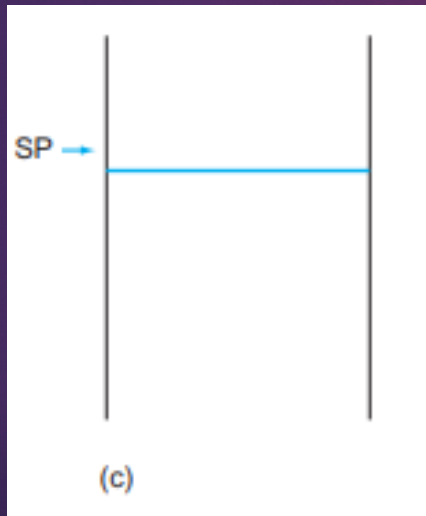
The values of the stack pointer and the stack **(a)** before, and **(b)** during, the procedure call.

* The stack pointer always points to the “top” of the stack, or the last word in the stack in this figure.

Example: Stacks – pop the values out

- Restore the 3 old values of registers we saved by 'popping' them from stack - **LIFO**.

```
lw x20, 0(sp)    # restore the value at the memory location currently pointed to by sp into register 'x20'.
lw x6, 4(sp)      # restore the value at sp+4 (4 Bytes higher from sp) into register 'x6'.
lw x5, 8(sp)      # restore the value at sp+8 into register 'x5'.
addi sp, sp, 12   # adjusts sp back by 12 Bytes 'deallocating' the space that was previously reserved for storing the 3 registers.
```



The values of the stack pointer and the stack (c) after the procedure call.

Environment Calls

- An **environment call (ecall)** is a special instruction used to request services from the environment or operating system.
- **ecall** is used to perform certain pre-defined system-level tasks, such as printing values, reading input, or terminating a program.

ID (a0)	Name	Description
1	print_int	prints integer in a1
4	print_string	prints the null-terminated string whose address is in a1
9	sbrk	allocates a1 bytes on the heap, returns pointer to start in a0
10	exit	ends the program
11	print_character	prints ASCII character in a1
17	exit2	ends the program with return code in a1

Environment Calls

- To print (int string, ASCII char etc.)

li a1, 42 #Load the integer value 42 into a1 (the value to print)

li a0, 1 #Set a0 to 1 to indicate a **print integer** ecall

ecall #Make the system call to print the integer

ID (a0)	Name	Description
1	print_int	prints integer in a1

- a0=x10 (integer, e.g.: if you want print >> int 1, string 4, ASCII 11)
- a1=x11 (Data you want to print >> 42 in this case)